COSC2674/2755 (PIoT) Semester 1, 2026
Individual Assignment Specification

Marks allocated:	35 (worth **35%** of the total score)
Deadline:	Thursday 9.04.2026 11:59 pm
Submit via:	Canvas and GitHub
Work mode:	**individual**
Submission format:	Repo **URL** on Canvas

1.1 Aim

The aim of this assignment is to write a small IoT application using Raspberry Pi and Sense HAT in Python language. Some of the tasks of this assignment will require self-exploration and research, you will not find the answer in lectures notes and/or tutorials.

You must attend a demo session to get assignment 1 marked. A schedule and a booking document will be published soon. You must submit the assignment prior to demo. **No submission → No demo → No marks**.

1.2 Important | *please read this section*

You must adhere to the following requirements:

a. Raspberry Pi model 5, 4 or 3 may be used.

b. You must use Python >= 3.5 to complete the tasks. Older versions must not be used.

c. Whether working individually or in a group, you must use *GitHub* for version control.

d. Try use object-oriented code if possible. Procedural code to fetch 50% scores if object-oriented option is available.

e. Your files must be under correct folders, e.g., all files in Task A must be under a folder named "TaskA". Otherwise, 50% penalty will be applied.

f. In the demo session, the marker will ask you to simply modify your code logic, e.g., adding random numbers or functions, for testing purpose. Failed to satisfy the modification will introduce 20~80% penalty.

1.3 Tasks

Coding Tasks (Please note that edge cases in all tasks should be considered, even not mentioned. You should design the mechanisms to handle them.)

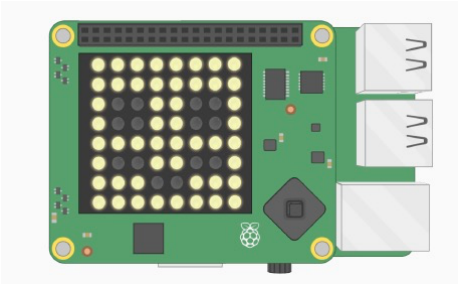
In this course we cover SQLite & MySQL, if you want to use another database, please feel free to do so. Use of Cloud databases is not allowed for assignment 1, you must use a local database installed on your Raspberry Pi. Except database, you can only use Python.

[Task 1] (10 marks) Create a folder named "Task 1".
(1) Create a Python script named `moodAnimator.py`. Your goal is to create 5 original animated emoji faces, each expressing a different emotion (e.g., happy, sadness, anger, …). Each face must consist of a multi-frame animation, with a minimum of 3 distinct frames per emoji, and at least 4 colours per frame.

Users must be able to navigate through the emoji set using the joystick:

- Right press: move to the next animated emoji
- Left press: go to the previous one
- Middle press: pause or resume the animation

Your program should continuously play the animation for the current emoji unless paused. Implement a 3-second rate-limit on presses: ignoring any continuous presses within this 3-second time window to ensure only one action occurs per interval. Additionally, implement idle timeout detection: if no joystick input occurs for 15 seconds, the display should dim the LED brightness and enter a "sleep mode" (e.g., show a low-glow neutral face). On any joystick input, the system should wake and resume the last selected emoji and frame.



(2) Develop a second Python script named `tiltEmotions.py`. This program must read orientation data from the Sense HAT's pitch, roll, and yaw angles and use it to classify the current "mood" state of the device (from (1)). Based on defined angle thresholds, each orientation quadrant or pose should correspond to an emotion. For example:

- Tilt right → MoodEmo1
- Tilt left → MoodEmo2
- Tilt forward → MoodEmo3
- Tilt back → MoodEmo4
- Flat → MoodEmo5

Instead of a static display, each mood must display a brief animated sequence, possibly including subtle changes in expression or colour. The display should update only when a new orientation zone is detected, to avoid flickering.

Also, implement joystick pause/resume control, and add logic that detects rapid flipping (e.g., roll change > 60° within 0.5s) to trigger a new special reaction, i.e., MoodEmo6, which is not implemented in (1).

[Task 2] (5 marks) Create a folder named "Task 2". Create a Python program using the Raspberry Pi Sense HAT that displays information about number calculations on the LED matrix when the Sense HAT's joystick is pressed.

- Default value is x = 16; always show the value of x on the LED matrix
- Press with the up direction: increase the value of x by 1
- Press with the down direction: decrease the value of x by 1
- Press with the left direction: assign the square of x to x
- Press with the right direction: assign the square root of x to x
- Press in the middle: reset x by the default value

[Task 3] (10 marks) Create a folder named "Task 3". The objective is to monitor device orientation and environmental parameters, such as temperature and humidity, and provide real-time feedback based on user-defined thresholds.

Begin by creating a structured configuration file called `enviro_config.json`. This file must define threshold categories for each monitored parameter: temperature, humidity, pressure, and orientation. For environmental variables, include minimum and maximum acceptable values or comfortable ranges—for instance, temperature between **15°C and 24°C**, and humidity between **30% and 65%**. For orientation, the config should specify allowable bounds for pitch, roll, and yaw, for example, pitch and roll should not exceed **±30°**, and yaw should **remain** within a certain sector to ensure the device remains correctly aligned. Each section of the config file must include structured data with keys indicating acceptable ranges, and your script must validate these entries. If invalid or malformed, the program must terminate cleanly with descriptive error feedback.

The core script, named `SensorMonitor.py`, should run continuously, polling the Sense HAT every 10 seconds. It must gather data for all parameters and adjust the temperature reading with a calibration offset (e.g., subtracting 2°C to correct for internal heat). Each reading must then be evaluated against the configuration file and assigned a classification. For example, temperature readings may be labeled “Low”, “Comfortable”, or “High”, and orientation angles may be flagged as “Aligned” or “Tilted” based on whether they remain within the predefined pitch/roll/yaw limits.

Each reading, along with its status and a timestamp, should be logged to a local database (such as **SQLite** or **MySQL**) called `envirotrack.db`. Your script should automatically create the database and its tables if they do not exist. Each log entry must include the raw sensor values, their respective categories (e.g., temperature level, orientation status), and the measurement time. Ensure your database interactions are secure and reliable by using parameterized queries.

The system should also provide real-time feedback on the Sense HAT’s LED matrix. The display should alternate every 5 seconds among different sensor outputs—for example: T:22 for temperature, H:55 for humidity, and P:12/R:12/Y for pitch/roll/yaw. Use a color-coded scheme to represent each classification: green for normal/comfortable ranges, red for high, blue for low, and amber for “Tilted” orientation alerts.

Furthermore, the system should support joystick control. A press of the joystick should pause all sensor reading and LED updates, holding the last displayed status. Pressing it again should resume the normal sensing and display cycle. This behavior must be implemented in a non-blocking manner using event listeners to ensure responsiveness during execution.

[Task 4] (5 marks) Create a folder named "Task 4". Read data from the database in **Task 3**, and use 2 different Python data visualisation libraries to create 2 images (e.g., png files). You will need to do some research of your own.

You will now need to think:
- With the data that you have in the database, what kind of data visualisation graph should be used? (Scatter plots, Bar charts and Histograms, Line plots, Pie charts, etc.)
- What should be represented in the above graph? Why did you make that decision?
- Python supports numerous data visualisation libraries. Which two libraries did you narrow it down to?

Create a python file called `analytics.py` that creates the above-mentioned images.

For Master students only: in addition to creating the images, write a text file called `analytics.txt` comparing the different techniques used:
- Questions stated above and,
- an analytical comparison of the libraries used, advantages/disadvantages, simplicity/complexity, flexibility/configurations, anything you may think is necessary,
- Formal references if needed.

Please note that this sub-question will introduce deduction (0-5 marks) only if the answer is unsatisfied.

Professionalism and Teamwork-related tasks

[Task 5] (5 marks) Professional use GitHub for version control and how the code has been developed over time. Please read the assignment rubrics for details.

- Use object-oriented python code if possible.
- Note: You can add more python files if necessary.
- Note: necessary comments must be included in your files for explanations.
- Note: necessary information, such as student id and name, must be included in README file.

1.4 What and how to submit?

Submit your GitHub repository **URL** in Canvas. The repository should contain all the files of this assignment.

1.5 Late submission and Extension

a. A penalty of 10% per day (*which is 3.5 marks per day in case of this assignment*) of the total marks will apply for each day late, including both weekend and weekdays.

b. Extension requests should only be emailed to the lecturer (xiaoyu.xia@rmit.edu.au) before your (extended) deadline.

1.6 Plagiarism

All assignments will be checked with **plagiarism-detection software** and **AI detect tools**; any student found to have plagiarised will be subject to disciplinary action. Plagiarism includes:

- Submitting work that is not your own or submitting text that is not your own.

- Allowing others to copy your work via email, printouts, social media, GitHub, etc…
- Posting assignment questions (in full or partial) on external technical forums.
- Copying work from/of previous/current semester students.
- Sending or passing your work to your friends.
- Posting assignment questions on technical forums to get them solved.

A disciplinary action can lead to:

- A meeting with the disciplinary committee.
- A score of zero for the assignment.
- A permanent record of copying in your personal university records and/or
- Expulsion from the university, in some severe cases.

All plagiarism will be penalised. There are no exceptions and no excuses. You have been noticed.